Metadata Versioning for Persistent Identifiers

Doan, Triet
triet.doan@gwdg.de
Gesellschaft für wissenschaftliche Datenverarbeitung mbH Göttingen, Germany

Böhm, Jana
jana.boehm@gwdg.de
Gesellschaft für wissenschaftliche Datenverarbeitung mbH Göttingen, Germany

Bingert, Sven
sven.bingert@gwdg.de
Gesellschaft für wissenschaftliche Datenverarbeitung mbH Göttingen, Germany

Introduction

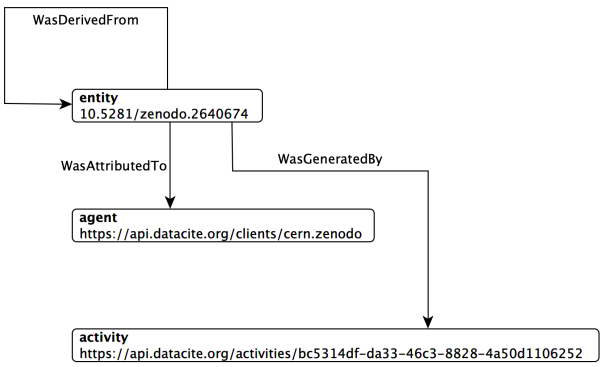
FAIR principles have become a best practice in research data management. It stands for Findable, Accessible, Interoperable, and Reusable (Wilkinson, et al., 2016). Whenever researchers publish their data, e.g. papers, data sets, or any kind of Digital Objects (DOs), it is strongly recommended that the published data should follow the FAIR principles. According to the FAIR Guiding Principles, a DO is findable when a) it is assigned to a globally unique Persistent Identifier (PID), and b) it is described with rich metadata. Suppose that a DO is already findable, which means it has a PID together with its metadata, our work focuses on condition R1.2. in the reusability guideline of FAIR, which says: "(meta)data are associated with detailed provenance". Provenance of a DO is information related to its creation or modification (e.g. creation time, modification time, by whom) (W3C, 2010). This information is crucial to determine if a DO is, e.g., trustworthy or not, thus enables reusability. To satisfy the requirement above, not only the DO itself, but also its metadata should be associated with detailed provenance. Therefore, it is necessary to track the changes of a DO's metadata.

In this paper, we propose two approaches to track the history of metadata in a PID, namely using Git and using only the PID system. The first approach exploits Git for metadata versioning, while the second one only relies on already available features of a PID system for the same purpose. For the proof-of-concept, we focus on the Handle System (Handle, n.d.), on which the Digital Object Identifier (DOI) system or Persistent Identifier for eResearch (ePIC)[1] PIDs are based. However, the presented concept should be applicable for other PID systems as well. Unlike the solutions from other systems, the proposed approach uses only available features from a PID system without relying on another third-party service or the need to build another history tracking system.

This paper is structured as follows: Section 1 presents the motivation and the necessity of the work. Related work is discussed in Section 2. Since our work focuses on the Handle System, a brief introduction about handles is presented in Section 3, which serves as a base for the further understanding of this paper. In Section 4, we describe our approaches in detail, together with their pros and cons. Section 5 shows a step-by-step instruction about retrieving a version for systems that implement one of our approaches. Finally, the conclusion is presented in Section 6.

Related Work

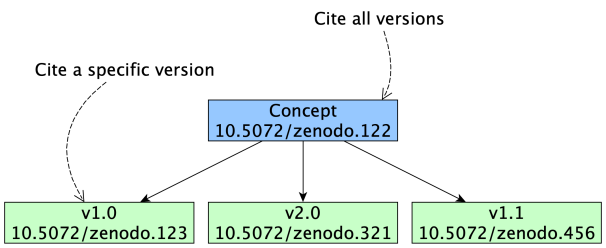
There is not much previous work about tracking provenance of metadata in PIDs. One example that we could find is the Provenance Service from DataCite (DataCite, n.d.). This service is responsible for tracking all activities that were performed on the metadata of their PIDs. This feature is available for all DOIs registered at DataCite since March 10, 2019. Figure 1 shows the data model of the Provenance Service from DataCite. This model follows the Prov Data Model (PROV-DM). PROV-DM is a conceptual data model created by W3C. It is a foundation for specifications about W3C provenance (Belhajjame, et al., 2013). As we can see from Figure 1, *entity* is a PID, whose metadata are changed. The changes are done by an *agent*, and described in an *activity*. This Provenance Service is available via the REST API of DataCite, specifically at the */activities* endpoint (DataCite, 2024).



Data model used by DataCite for their Provenance Service (FREYA, n.d.).

Since this is part of DataCite, the Provenance Service is only available for customers of DataCite. If researchers use other PID providers, it is not possible for them to have those features on their PIDs. Furthermore, this Provenance Service is a completely independent service, which offers more features on top of their PID service. Researchers or institutions, who do not use DataCite, might not have enough resources to build such a service for their need. Therefore, a more lightweight solution, which can be directly integrated into a running PID service, would be beneficial.

Another work that goes in the same direction is the DOI versioning feature of Zenodo (Nowak, et al., 2018). In Zenodo, each DO has a *Concept DOI*, which is the "master" DOI that represents all versions of this DO. Whenever a new version of the DO is uploaded, a new DOI is assigned to it and linked to the Concept DOI, as illustrated in Figure 2. With this model, researchers can cite a specific version of the DO, or they can cite all versions of the DO by simply citing the Concept DOI.

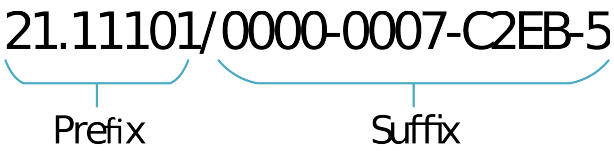


In Zenodo, each version of the DO has its own DOI. These DOIs are linked together via a Concept DOI. One can cite a specific version, or cite the Concept DOI, which represents all versions of the DO (Nowak, et al., 2018).

The approach from Zenodo is appropriate for versioning the DO, but not its metadata. Suppose that we apply this method to the metadata. Whenever there are changes in the metadata, a new version is created and, therefore, a new DOI for that version is generated. This leads to two problems. The first one is the explosion of the number of DOIs if metadata change frequently. The second problem comes from the conceptual viewpoint. Each DOI is an identifier of a DO. Whenever the DO changes, it becomes a new DO, thus gets a new identifier, DOI specifically. However, when only its metadata change, it does not make sense to assign a new identifier to the DO, since it is still the same object. For those reasons, we cannot adapt this approach to track the changes of metadata.

Handle System

Since the proof-of-concept is based on the Handle System, this section gives a brief overview about handles and how (meta)data are structured inside a handle. The Handle Software, which is an implementation of PIDs, is developed by the Corporation for National Research Initiatives (CNRI) (Handle, n.d.). It is used to manage handles and handle values. Every handle consists of two parts: its naming authority, otherwise known as its prefix, and a unique local name under the naming authority, otherwise known as its suffix. An example of a handle with prefix *21.11101* and suffix *0000-0007-C2EB-5* is shown in Figure 3. Thanks to the combination of prefix and suffix, any handle is globally unique within the context of the Handle System (Lannom, et al., 2003).



Example structure of a handle with the prefix 21.11101 and suffix 0000-0007-C2EB-5 . The prefix and suffix are separated by the ASCII character "/". A handle can contain any printable characters from the Universal Character Set of ISO/IEC 10646 (CNRI, 2018).

Inside a handle, users can create a collection of associated type-value pairs to store metadata of the related DO. Each pair has its own index, which is a unique key in this handle. Suppose we have a handle called *21.1234/MY-PAPER* for a published paper. An example of types and values for this handle is presented in Table 1. The first index stores the URL pointing to the location where the DO is located. The indices 2 to 4 contain some associated metadata, which is the author, the publication date as well as the institution.

Table 1 : Example handle, *21.1234/MY-PAPER* , which contains metadata about the published paper. The metadata are type-value pairs,

where values are simple atomic strings.

Index	Type	Value
1	URL	https://www.gwdg.de/paper.pdf
2	Author	Triet Doan
3	Publication Date	20.10.2023
4	Institution	GWDG

Note that in a handle, the same types can appear multiple times, as long as they are on different indices. Using terminologies from database design, in a table with three columns, *index*, *type*, and *value*, *index* is the primary key of the table. For example, handle *21.1234/ANOTHER-PAPER* (see Table 2) has another *Author* type at index 5 for the second author of the paper. When a handle is resolved, users are usually redirected to the specified URL. However, there is also an option to get back a list of type-value pairs as a result of the resolving process.

Table 2 : This table presents handle *21.1234/ANOTHER-PAPER* , which contains metadata about the published paper. The metadata of this paper show that it has two authors, which are stored at index 2 and 5, respectively.

Index	Type	Value
1	URL	https://www.gwdg.de/paper.pdf
2	Author	Triet Doan
3	Publication Date	20.10.2023
4	Institution	GWDG
5	Author	Jana Böhm

Proposed Approaches

Using Git

When talking about versioning, one usually thinks about Git, a well-known version control system. The idea of this approach is to store metadata of a PID in Git to fully utilize its feature for tracking changes. To do so, instead of putting metadata of a DO in a PID, as shown in Table 3, we can store them in a text file and push this file to a Git repository.

Table 3: Structure of the PID *21.1234/MY-PAPER* when its metadata is stored in GitHub. Instead of putting all metadata in the PID as was done in Table 1, this PID has another type called *METADATA*, whose value is a URL pointing to a file in GitHub. That file contains all metadata of this PID.

Index	Type	Value
1	URL	https://www.gwdg.de/paper.pdf
2	METADATA	https://github.com/my-project/main/metadata.json

Suppose that we use GitHub as a repository for metadata, Table 3 shows how the PID *21.1234/MY-PAPER* is structured. In this structure, the PID record contains only two entries. The first entry is the URL pointing to the location of the DO. The second one is the URL pointing to the location of the metadata text file. This text file consists of all relevant metadata fields for the PID. To ensure machine-readability, this file should be in plaintext format with structure (e.g. JSON), as shown in Figure 4. This file is put under a version control system (Git) and pushed to a Git repository. Consequently, all changes to this file will be recorded by Git and one can use all features of Git to navigate through the metadata history.

```
{
  "Author": "Triet Doan",
  "Publication Date": "20.10.2023",
  "Institution": "GWDG"
}
```

This is the content of the metadata.json file. This file contains all metadata of the DO, whose PID is 21.1234/MY-PAPER.

A clear advantage of this approach is the usage of Git for metadata provenance. Since Git is a well-

known and feature-rich version control system, one can expect to be able to perform various operations on data history, such as viewing a list of changes, checking the differences between versions, checking out another version, etc. Besides its obvious advantage, using Git also brings some drawbacks. If a PID provider wants to offer a possibility for users to see the history of their metadata on their websites, it might be challenging to implement since one needs to parse the Git history. Another disadvantage is how to organize data in a Git repository properly. Assume that GitHub is used. In a simple use case, we can have one repository per PID. However, this will make the number of repositories explode, since a PID provider can allocate millions of PIDs, which means millions of repositories need to be created. To circumvent this, one might try to put metadata files of different PIDs in one repository. Due to the fact that each repository has one history, this solution will mix the metadata history of many PIDs together. This leads to confusion and further techniques need to be applied (e.g. tagging) to distinguish the history of different PIDs. Last but not least, putting metadata in a JSON format eliminates the possibility to have duplicated types in a handle. Since all field names on the same level of a JSON must be unique, the JSON in Figure 4 cannot have two Author fields. To overcome this issue, one could combine index and type to form a JSON field name, or convert duplicated types into arrays.

Using only the PID system

For simple type-value pairs

Suppose we want to track the changes of metadata of the PID 21.1234/MY-PAPER shown in Table 1. The first step is to create another PID to store the history. We call it a changelog PID. This changelog PID is only created at the first moment when the metadata of the original PID are modified. Continuing with the example, this changelog PID is named *21.1234/MY-PAPER-METADATA-CHANGELOG*. Keep in mind that the suffix does not need to have a semantic meaning. Only for illustration, we used this example. Further work will evaluate if a naming convention for changelog PID should be developed. To know that our PID *21.1234/MY-PAPER* has a metadata changelog, another type can be added to this PID. For example, type *Metadata_history* will be added at index 100 with the value *21.1234/MY-PAPER-METADATA-CHANGELOG*. This type should be standardized to simplify future use and machine actionability. Possible solutions are adding it to the Handle Namespace (Lannom, et al., 2003) or using a type registry, which is currently developed in the EU funded project FAIRCORE4EOSC (FAIRCORE4EOSC, n.d.). Following the ideas of writing a changelog (Lacan, n.d.), a changelog PID stores all changes, which are actions performed on the metadata. By applying these changes in a correct order, one can reconstruct the metadata of a PID to an exact state in the past. We define three kinds of actions: *add*, *update*, and *delete*.

Table 4 shows the content of the PID *21.1234/MY-PAPER-METADATA-CHANGELOG*, which has 4 entries, starting from index 1 to index 4. The PID which owns the corresponding changelog is always stored at index 1, while changes are recorded starting from index 2. The index number acts as the order of changes. It means the change at index 3 happened after the change at index 2. Note that the value at index 2 is the initial state of the metadata. It assumes that one starts with an empty PID, then metadata are added in as a step in the PID generation process.

Table 4: This is the content of the PID *21.1234/MY-PAPER-METADATA-CHANGELOG*. This PID is a changelog of the PID *21.1234/MY-PAPER*. It stores changes which were applied to the metadata of the PID *21.1234/MY-PAPER*. The owner of this changelog PID is specified at index 1. Changes are described from index 2 on. Each *value* is a JSON object, whose schema can be found at https://doi.org/10.25625/XWKW1R .

Index	Type	Value
1	Owner	21.1234/MY-PAPER

2	Changelog	{
		"version": "0.0.1",
		"timestamp": "2023-11-01T10:48:39+00:00",
		"author": "tdoan",
		"changelog": [
		{
		"action": "add",
		"index": 1,
		"type": "URL",
		"value": "https://www.gwdg.de/my-paper.pdf"
		},
		{"action": "add", "index": 2, "type": "Author", "value": "Triet Doan"},
		{"action": "add", "index": 3, "type": "Submission date", "value":
		↪ "20.10.2023"},
		{"action": "add", "index": 4, "type": "Institution", "value": "GWDG"}
		]
		}

3	Changelog	{
		"version": "0.1.0",
		"timestamp": "2023-11-05T08:50:10+00:00",
		"author": "tdoan",
		"changelog": [
		{
		"action": "add",
		"index": 5,
		"type": "Reviewer",
		"value": "Mr. X"
		},
		{
		"action": "update",
		"index": 3,
		"old_value": "20.10.2023",
		"value": "22.10.2023"
		}
		]
		}

4	Changelog	{
		"version": "1.0.0",
		"timestamp": "2023-12-07T14:15:07+00:00",
		"author": "jboehm",
		"changelog": [
		{
		"action": "delete",
		"index": 4,
		"type": "Institution",
		"value": "GWDG"
		}
		]
		}

To make it machine-readable, each value in the changelog PID is a JSON object (except the value at index 1). This JSON has four keys: *version*, *timestamp*, *author*, and *changelog*. As the names suggest, *version* is the new version number, *timestamp* is the time when the changes were made, and *author* is the one who did it. Meanwhile, *changelog* contains a list of changes that were applied. Each change starts with an *action*, which is one of the three actions defined above. For example, in Table 4, we can see that at the beginning (index 2), four metadata entries were added by *tdoan* at the specified timestamp, which form the initial state of the PID 21.1234/MY-PAPER.

Depending on the action, one needs to provide information to the changelog accordingly. An *add* or *delete* action requires all three *index*, *type*, and *value* to be provided for each change in the changelog. However, for an *update* action, *type* is not necessary, but *index*, *value*, and *old_value* are required. By storing not only new values but also the old ones, the history can be traversed in both directions. To reach a specified version, one can either start from the beginning and continue applying changes, or start from the latest version and then rollback.

In a changelog PID, version numbers in *version* fields follow the Semantic Versioning specification (Preston-Werner, n.d.). In this syntax, a version number has the format *MAJOR.MINOR.PATCH*. The specification has already explained when to increase which number. To apply it to our approach, an *update* action is a patch, an *add* action is a minor change, and *delete* is a major change. In software release management, whenever a feature is removed, the software is not backward compatible anymore. Therefore, after a delete action, the new version must be a major release.

For JSON values

The presented approach is for PIDs which organize their metadata as type-value pairs, where values are atomic strings. In practice, there are cases where a value of a metadata field is in JSON format. The PID shown in Table 5 is an example for this use case. It shows a PID, *21.1234/JSON-METADATA*, which was assigned to a person. As we can see, the metadata of this person are all stored in index 1 as a string in JSON format.

Table 5 : This example PID, *21.1234/JSON-METADATA* , has JSON as a value of its metadata field.

Index	Type	Value
1	Person	`{ "name": "John Smith", "age": 25, "address": { "street": "Baker Street", "city": "London" }, "hobbies": [ { "name": "Table tennis", "level": "advanced" }, { "name": "Chess", "level": "beginner" } ] }`

Suppose that after some time, that person has moved to another city, and starts playing guitar as a new hobby. So, the metadata in that PID must be updated. The approach is almost the same as what was described for atomic strings. It means that a changelog PID must be created with proper *Owner* type and version syntax, and changes need to be described in the JSON format. However, in this case, since metadata are stored in a JSON object, the structure of each element in the *changelog* must be adjusted properly. Table 6 shows how the changelog is initiated and how changes on *city* and *hobbies* are documented. To identify the changed fields in a *JSON* object, *JSONPath* is used as the value for the *path* field. JSONPath is like XPath, but for JSON (Gössner, 2007). It is a standardized way to describe how to access to a field in a JSON object. By using JSONPath to identify the updated field, we can fully ensure machine-readability of the changelog PID. Same as the other case, when an update action is performed, *old_value* field is required.

Table 6: This changelog PID shows two changes to the PID *21.1234/JSON-METADATA* described in Table 5. The author has changed the *city* and added a new *sport*. JSONPath is used to identify the changed fields in a JSON object.

Index	Type	Value
1	Owner	21.1234/JSON-METADATA
2	Changelog	`{ "version": "0.0.1", "timestamp": "2023-10-01T09: "author": "tdoan", "changelog": [{ "action": "add", "index" "value": { "name": "John Smith", "age": 25, "address": { "street": "Baker Str "city": "London" }, "hobbies": [{ "name": "Table ten "level": "advanced }, { "name": "Chess", "level": "beginner }] } } }] }`
3	Changelog	`{ "version": "0.1.0", "timestamp": "2023-12-0 "author": "tdoan", "changelog": [ { "action": "update", "index": 1, "path": "$.address. "old_value": "Londo "value": "Liverpool }, { "action": "add", "index": 1, "path": "$.hobbies" "value": { "name": "Guitar", "level": "beginne } } ] }`

The schema for the *changelog* type can be found at https://doi.org/10.25625/XWKW1R . This schema includes both cases which were discussed above. By using this schema, PID providers have a possibility to validate the changelog PID to make

sure that everything is correct before allowing the modification to take place.

The clear benefit of this approach is its simplicity. There is no need to build another service or use any third-party program to track the provenance of the DO's metadata. In addition, changes are described in a machine-readable way. This allows the data to be processed by any systems, independent of the programming languages they use. Furthermore, by storing only the changes instead of the whole new state, redundancy is avoided, thus greatly reduces the size of a changelog PIDs. Last but not least, both new values and old values are stored in the changelog. This allows the history to be traversed in both directions: either from the beginning to the specified version, or from the latest version back to another version in the past.

Besides its advantages, this approach has a limitation which need to be taken into consideration at the implementation phase. It is considerable that the size of the changelog PID only grows and will never become smaller. Therefore, if there is a limit in PID size, the system needs to paginate the changelog. This can be easily achieved by creating a new PID for the new page, then linking these PIDs properly with the *next* and *previous* references.

Retrieve a version

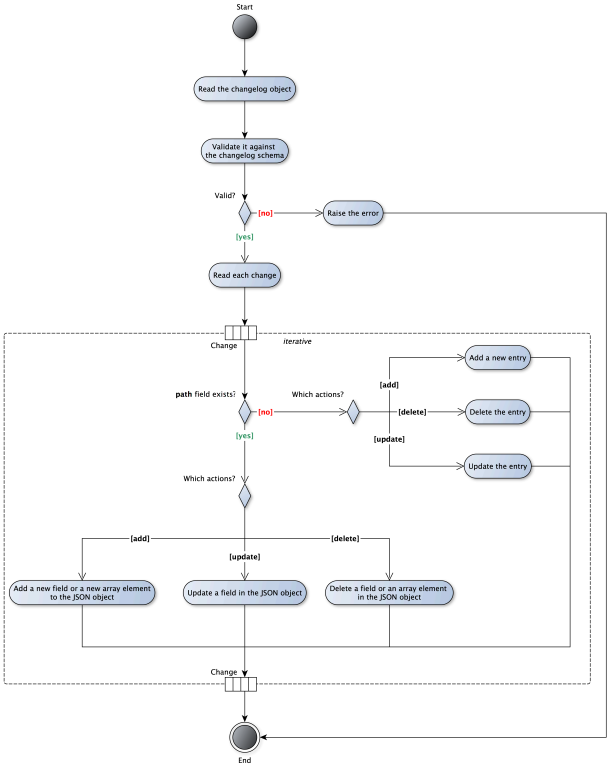
When users resolve a PID to see its metadata, the latest version is always returned by default. However, there are cases where previous versions are needed. To support that feature, there must be a systematic way to retrieve a specific version by reading the changelog.

Suppose that a PID provider applies the approach described in Section 4.2 to track changes of metadata of their PIDs. To retrieve an older version, the system starts with an empty record, then goes through all changes up to the desired version and modifies the record accordingly. The detailed steps about how to read a changelog are illustrated as an activity diagram in Figure 4.

When reading a changelog, the system must first validate it. If the JSON does not conform to the schema, the system must immediately abort and return with an error. This validation step is vital, since it ensures that every valid change has all required fields, and the combination of fields makes sense. For each change, as presented in Figure 4, if the *path* field exists, the system assumes that metadata is stored as a JSON object, similar to the one shown in Table 5. Therefore, the specified action will be performed on the JSON object at the specified index. On the other hand, without the *path* field, the action will be done on the whole entry level. With this algorithm, although the system receives the same action, e.g. *add*, it can distinguish if a new field should be added to the JSON object, or a new entry should be added to the record. For the rollback, the same workflow is used with small adjustments. For every *add* action, *delete* is done instead, and vice versa. For *update*, *old_value* is used to replace *value*.

Conclusion

Research data management should follow the FAIR principles. To ensure reusability, not only the DO, but also its metadata should be “associated with detailed provenance”. Also, the FAIR Digital Object (FDO) framework (FDO, n.d.), which is currently under debate, might have the requirement to maintain the provenance of specific information stored in a PID. In this work, two approaches for tracking changes of metadata are presented, namely using Git and using only the PID system. Each approach has its advantages and disadvantages. Although the Git-approach offers a variety of features, it is not scalable. Therefore, it is only appropriate for individual users, who do not own too many PIDs. The second approach, which only utilizes features of a PID system, is more flexible. It requires some implementation and does not offer many features like Git. However, it is scalable with respect to the number of PIDs. For that reason, we recommend PID providers to use this approach when they want to offer the metadata versioning feature.



This activity diagram shows how the system reads a changelog. It starts with the validation, then it parses each change and performs the described action on a record to restore the desired version.

Notes

1. ePIC is a European consortium that provides Handle based PID services for the European Research Community. For more information, please visit https://www.pidconsortium.net/

Bibliography

Belhajjame, K. et al., 2013. PROV-DM: The PROV Data Model. W3C Recommendation, Volume 14, pp. 15-16.

CNRI, 2018. Handle.net Technical Manual. [Online] Available at: https://hdl.handle.net/20.1000/113 [Accessed 8 April 2025].

DataCite, 2024. Get a JSON API result of activities. [Online] Available at: https://support.datacite.org/reference/get_activities [Accessed 17 January 2024].

DataCite, n.d. Tracking metadata provenance. [Online] Available at: https://support.datacite.org/docs/tracking-provenance [Accessed 16 January 2024].

FAIRCORE4EOSC, n.d. EOSC Data Type Registry (DTR). [Online] Available at: https://faircore4eosc.eu/eosc-core-components/eosc-data-type-registry-dtr [Accessed 8 April 2025].

FDO, n.d. FAIR Digital Objects Forum. [Online] Available at: https://fairdo.org/ [Accessed 30 January 2024].

FREYA, n.d. Provenance Services. [Online] Available at: https://project-freya.eu/provenance.html [Accessed 08 April 2025].

Gössner, S., 2007. JSONPath - XPath for JSON. [Online] Available at: https://goessner.net/articles/JsonPath/ [Accessed 12 January 2024].

Handle, n.d. Handle.Net Registry. [Online] Available at: https://www.handle.net [Accessed 15 January 2024].

Lacan, O., n.d. keep a changelog. [Online] Available at: https://keepachangelog.com/en/1.1.0/ [Accessed 9 January 2024].

Lannom, L., Boesch, L. C. B. P. & Sun, S., 2003. Handle System Overview. [Online] Available at: https://www.rfc-editor.org/info/rfc3650 [Accessed 8 April 2025].

Lannom, L., Reilly, S. & Sun, S., 2003. Handle System Namespace and Service Definition. [Online] Available at: https://www.rfc-editor.org/info/rfc3651 [Accessed 8 April 2025].

Nowak, K., Ioannidis, A., Bigarella, C. & Nielsen, L. H., 2018. Zenodo. [Online] Available at: https://doi.org/10.5281/zenodo.1256592 [Accessed 8 April 2025].

Preston-Werner, T., n.d. Semantic Versioning 2.0.0. [Online] Available at: https://semver.org/ [Accessed 10 January 2024].

W3C, 2010. What is Provenance. [Online] Available at: https://www.w3.org/2005/Incubator/prov/wiki/What_Is_Provenance [Accessed 3 January 2024].

Wilkinson, M. D. et al., 2016. The FAIR Guiding Principles for scientific data management and stewardship. Scientific data, 3(1), pp. 1-9.